CLASE PRÁCTICA 4

Patrones de diseño orientado a objetos

Ejercicios 8 y 9

Sistema de Integración de Pagos y Sistema de Notificaciones de Bolsa

---

Equipo: LA ESCALONETA

Integrantes y distribución del trabajo

Integrante	Grupo	Preguntas e incisos
Ramón Hernández Camacho	C311	8, incisos a) y b)
Marlon R. Cuellar García	C312	8, inciso c)
Leandro Márquez Blanco	C311	8, inciso d)
Raciel Pupo Santos	C312	9, incisos a) y b)
Charly Blanco Sánchez	C311	9, incisos c) y d)

Repositorio del proyecto

https://github.com/CuellarMarlon/timeblock

Octubre de 2026

Índice

Ejercicio 8 — Sistema de Integración de Pagos **3**

8.aIdentificación del Patrón, Justificación y Análisis de Alternativas **3**

8.bImplementación del Sistema en C# **4**

8.b.1.Interfaz Objetivo (Target) . . . 4

8.b.2.Sistemas Legacy y Nuevo Servicio (Adaptees) . . . 4

8.b.3.Adaptadores Concretos (Adapters) . . . 5

8.b.4.Cliente y Programa de Prueba . . . 6

8.cAnálisis: Diferencias entre Adapter y Bridge **7**

8.c.1.¿Qué diferencia hay entre Adapter y Bridge? . . . 7

8.c.2.¿Cuándo utilizar cada uno? . . . 7

8.c.3.¿Qué sucede si el nuevo servicio cambia su API? . . . 8

8.dIntegración con Factory Method, Proxy y Decorator **8**

8.d.1.Factory Method: creación de adaptadores . . . 9

8.d.2.Proxy: almacenamiento en caché de respuestas de pago . . . 10

8.d.3.Decorator: auditoría de transacciones . . . 11

8.d.4.Integración de los tres patrones . . . 12

8.d.5.Relación entre los patrones utilizados . . . 14

Ejercicio 9 — Sistema de Notificaciones de Bolsa **15**

9.aIdentificación del patrón, justificación y análisis de alternativas **15**

9.a.1.Patrón seleccionado: Observer . . . 15

9.a.2.Justificación . . . 15

9.a.3.Clases que intervienen . . . 15

9.a.4.Relaciones y diagrama de clases . . . 16

9.a.5.Flujo de una notificación . . . 16

9.a.6.¿Por qué no Mediator? . . . 16

9.a.7.¿Por qué no polling (consulta periódica)? . . . 17

9.bImplementación con cinco acciones y mil clientes **17**

9.b.1.Estructura . . . 17

9.b.2.Cómo se cumplen los requisitos . . . 17

9.b.3.Decisiones de diseño . . . 18

9.b.4.Código fuente . . . 18

9.b.5.Salida de la ejecución . . . 22

9.cGestión de fallos, ejecución asíncrona y comportamiento de Observer **23**

9.c.1.Las tres preguntas del inciso . . . 23

9.c.2.¿Observer es síncrono o asíncrono? . . . 23
9.c.3.Problemas de la versión síncrona . . . 23
9.c.4.¿Qué pasa si un cliente no puede ser notificado? . . . 24
9.c.5.¿Cómo evitar bloquear el hilo principal? . . . 24
9.c.6.Qué se gana con este cambio . . . 26
9.c.7.Errores comunes . . . 26
9.c.8.Resumen del inciso c) . . . 26

9.dRendimiento con un millón de clientes y escalabilidad 27

9.d.1.Planteamiento del problema . . . 27
9.d.2.Modelo de costo . . . 27
9.d.3.Observer clásico síncrono . . . 27
9.d.4.Observer con Thread Pool . . . 27
9.d.5.Reducir N a K con un índice de umbrales . . . 28
9.d.6.Reactive Streams . . . 29
9.d.7.Tabla comparativa: cómo cambia el orden de complejidad . . . 30
9.d.8.Resumen del inciso d) . . . 30

Ejercicio 8 — Sistema de Integración de Pagos

8.a. Identificación del Patrón, Justificación y Análisis de Alternativas

Patrón Seleccionado: Adapter (Estructural)

El patrón de diseño más adecuado para este problema es Adapter. Este patrón permite convertir la interfaz de una clase existente en otra interfaz que el cliente espera, posibilitando que clases con interfaces incompatibles puedan colaborar de manera transparente sin modificar su código fuente original.

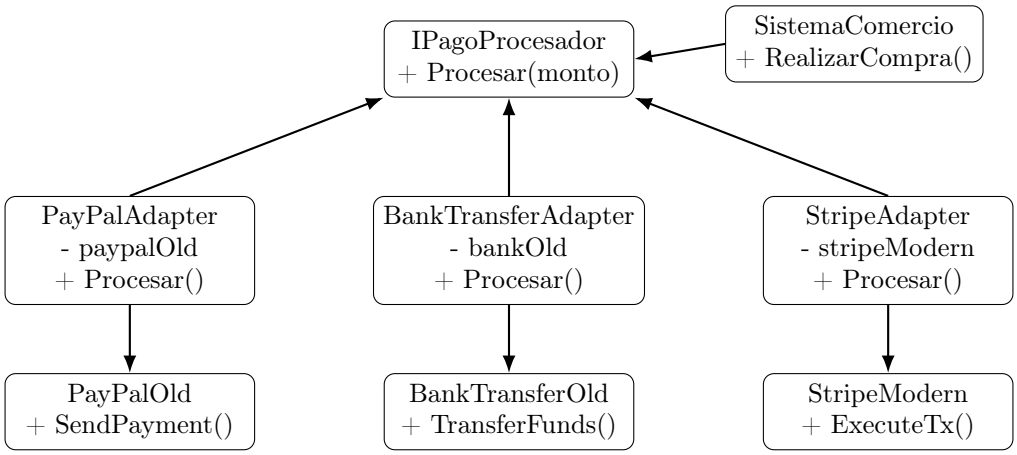
Justificación

1. **Incompatibilidad de contratos:** El sistema cuenta con clientes que realizan pagos esperando una interfaz uniforme. Los servicios existentes (PayPalOld y BankTransferOld) y el nuevo servicio (StripeModern) poseen nombres de métodos, firmas y estructuras de parámetros completamente heterogéneos.
2. **Imposibilidad de modificar el código existente:** Reescribir de inmediato todo el sistema cliente no es viable. Adapter actúa como un traductor intermedio que recibe las llamadas en el formato estándar del cliente y las delega al servicio correspondiente en el formato específico de su API.
3. **Cumplimiento de principios SOLID:** Permite mantener el principio Open/Closed (abierto a extensión, cerrado a modificación), ya que integrar nuevos proveedores en el futuro solo requerirá crear un nuevo adaptador sin modificar el cliente.

Explicación de la propuesta de solución (Clases y Relaciones)

1. **Interfaz Target (IPagoProcesador):** Define el contrato único y estandarizado que espera el cliente (método Procesar(decimal monto)).
2. **Adaptees (PayPalOld, BankTransferOld, StripeModern):** Clases concretas que realizan el procesamiento real del pago pero cuyas APIs son incompatibles entre sí.
3. **Adapters (PayPalAdapter, BankTransferAdapter, StripeAdapter):** Clases concretas que implementan IPagoProcesador y contienen una referencia privada a su respectivo Adaptee. Traducen la llamada estándar al método específico del proveedor.
4. **Cliente (SistemaComercio):** Invoca las operaciones de pago dependiendo exclusivamente de la abstracción IPagoProcesador, manteniéndose totalmente desacoplado de las implementaciones concretas.

Estructura de relaciones (Diagrama en texto)



¿Por qué no usar Bridge o Facade?

1. **Por qué no usar Bridge:** Bridge se utiliza durante la fase inicial de diseño de una arquitectura para desvincular una abstracción de su implementación de modo que ambas puedan variar de forma independiente. En este problema, las clases concretas de pago ya existen previamente y son incompatibles. Adapter se aplica precisamente sobre clases ya existentes para resolver la incompatibilidad de sus interfaces.
2. **Por qué no usar Facade:** Facade busca definir una nueva interfaz simplificada para reducir la complejidad de un sistema entero compuesto por múltiples clases interconectadas. En cambio, Adapter no busca simplificar una complejidad estructural, sino adaptar una interfaz existente a un contrato estándar ya prefijado para permitir la sustitución transparente de objetos.

8.b. Implementación del Sistema en C#

8.b.1. Interfaz Objetivo (Target)

```
public interface IPagoProcesador
{
    bool Procesar(decimal monto);
}
```

8.b.2. Sistemas Legacy y Nuevo Servicio (Adaptees)

```
// Legacy 1
public class PayPalOld
{
    public void SendPayment(double amount)
    {
        Console.WriteLine(
            $"[PayPal Legacy] Pago de ${amount:F2} procesado con éxito.");
    }
}

// Legacy 2
public class BankTransferOld
{
    public bool TransferFunds(decimal total, string currency)
```

```
    {
        Console.WriteLine(
            $"[BankTransfer Legacy] Transferencia por {currency} {total:F2} ejecutada.");

        return true;
    }
}

// Nuevo Servicio
public class StripeRequest
{
    public long AmountInCents { get; set; }
    public string Currency { get; set; } = "USD";
}

public class StripeModern
{
    public string ExecuteTransaction(StripeRequest request)
    {
        decimal monto = request.AmountInCents / 100m;

        Console.WriteLine(
            $"[Stripe Modern] Transacción realizada por ${monto:F2} ({request.Currency}).");

        return "ST_SUCCESS_2026";
    }
}
```

8.b.3. Adaptadores Concretos (Adapters)

```
// Adaptador para PayPal Legacy
public class PayPalAdapter : IPagoProcesador
{
    private readonly PayPalOld _payPalOld;

    public PayPalAdapter(PayPalOld payPalOld)
    {
        _payPalOld = payPalOld;
    }

    public bool Procesar(decimal monto)
    {
        _payPalOld.SendPayment((double)monto);
        return true;
    }
}

// Adaptador para Bank Transfer Legacy
public class BankTransferAdapter : IPagoProcesador
{
    private readonly BankTransferOld _bankTransferOld;

    public BankTransferAdapter(BankTransferOld bankTransferOld)
    {
        _bankTransferOld = bankTransferOld;
    }

    public bool Procesar(decimal monto)
    {
        return _bankTransferOld.TransferFunds(monto, "USD");
    }
}

```

```
// Adaptador para Stripe Moderno
public class StripeAdapter : IPagoProcesador
{
    private readonly StripeModern _stripeModern;

    public StripeAdapter(StripeModern stripeModern)
    {
        _stripeModern = stripeModern;
    }

    public bool Procesar(decimal monto)
    {
        var request = new StripeRequest
        {
            AmountInCents = (long)(monto * 100),
            Currency = "USD"
        };

        string status = _stripeModern.ExecuteTransaction(request);

        return status.StartsWith("ST_SUCCESS");
    }
}
```

8.b.4. Cliente y Programa de Prueba

```
public class SistemaComercio
{
    private readonly IPagoProcesador _procesadorPago;

    public SistemaComercio(IPagoProcesador procesadorPago)
    {
        _procesadorPago = procesadorPago;
    }

    public void RealizarCompra(decimal monto)
    {
        Console.WriteLine(
            $"Iniciando proceso de compra por ${monto:F2}...");

        bool resultado = _procesadorPago.Procesar(monto);

        if (resultado)
            Console.WriteLine("-> Compra completada exitosamente.\n");
        else
            Console.WriteLine("-> Error al procesar la compra.\n");
    }
}

public class Program
{
    public static void Main()
    {
        Console.WriteLine(
            "=== SISTEMA DE INTEGRACIÓN DE PAGOS (ADAPTER PATTERN) ===\n");

        // 1. Uso con PayPal Legacy
        IPagoProcesador adaptadorPayPal =
            new PayPalAdapter(new PayPalOld());

        SistemaComercio cliente1 =
            new SistemaComercio(adaptadorPayPal);

```

```
        cliente1.RealizarCompra(150.00m);

        // 2. Uso con Transferencia Bancaria Legacy
        IPagoProcesador adaptadorBanco =
            new BankTransferAdapter(new BankTransferOld());

        SistemaComercio cliente2 =
            new SistemaComercio(adaptadorBanco);

        cliente2.RealizarCompra(500.50m);

        // 3. Uso con Stripe Moderno
        IPagoProcesador adaptadorStripe =
            new StripeAdapter(new StripeModern());

        SistemaComercio cliente3 =
            new SistemaComercio(adaptadorStripe);

        cliente3.RealizarCompra(99.99m);
    }
}
```

8.c. Análisis: Diferencias entre Adapter y Bridge

8.c.1. ¿Qué diferencia hay entre Adapter y Bridge?

Adapter y Bridge son dos patrones de diseño estructurales que utilizan la composición para relacionar diferentes clases. Sin embargo, aunque pueden parecer similares, resuelven problemas distintos.

Adapter tiene como objetivo hacer compatibles dos interfaces que originalmente no pueden trabajar juntas. Para ello, introduce una clase intermedia que traduce las operaciones de una interfaz a otra. Generalmente se utiliza cuando necesitamos integrar componentes que ya existen y no queremos modificar su implementación original.

Por otra parte, **Bridge** tiene como objetivo separar una abstracción de su implementación, permitiendo que ambas puedan evolucionar independientemente. Su estructura evita que una jerarquía de clases crezca demasiado cuando existen diferentes variantes de abstracciones e implementaciones.

Las principales diferencias son:

Criterio	Adapter	Bridge
Objetivo	Hacer compatibles interfaces diferentes.	Separar una abstracción de su implementación.
Momento de aplicación	Generalmente cuando ya existen clases que deben integrarse.	Principalmente durante el diseño de una arquitectura.
Problema que resuelve	Incompatibilidad entre interfaces.	Crecimiento de combinaciones entre abstracciones e implementaciones.
Relación principal	Un adaptador traduce las llamadas de una interfaz a otra.	Una abstracción delega operaciones en una implementación independiente.
Modificación de clases existentes	Normalmente no es necesario modificarlas.	Se diseña la estructura para que ambas partes puedan evolucionar independientemente.

8.c.2. ¿Cuándo utilizar cada uno?

- **Adapter:** cuando necesitamos integrar componentes existentes que poseen interfaces incompatibles. Por ejemplo, conectar PayPalOld y BankTransferOld con nuestro sistema de pagos sin

modificar sus implementaciones.

- **Bridge:** cuando estamos diseñando un sistema que tendrá diferentes abstracciones y diferentes implementaciones, y necesitamos que ambas dimensiones puedan evolucionar independientemente.

En nuestro problema, Adapter es la elección adecuada porque ya tenemos tres servicios de procesamiento de pagos con interfaces diferentes y necesitamos unificarlos bajo `IPagoProcesador`.

8.c.3. ¿Qué sucede si el nuevo servicio cambia su API?

Supongamos que StripeModern modifica su API y sustituye el método:

```
ExecuteTransaction(StripeRequest request)
```

por una nueva operación:

```
CreatePayment(PaymentIntent intent)
```

Este cambio no debería afectar directamente al resto del sistema.

La solución consiste en modificar o sustituir únicamente `StripeAdapter` para que traduzca las operaciones de nuestra interfaz `IPagoProcesador` a la nueva API de Stripe.

```
public class StripeAdapter : IPagoProcesador
{
    private readonly StripeModern _stripeModern;

    public StripeAdapter(StripeModern stripeModern)
    {
        _stripeModern = stripeModern;
    }

    public bool Procesar(decimal monto)
    {
        var intent = new PaymentIntent
        {
            Amount = (long)(monto * 100),
            Currency = "USD"
        };

        string status = _stripeModern.CreatePayment(intent);

        return status.StartsWith("ST_SUCCESS");
    }
}
```

En este caso, el cliente `SistemaComercio` continúa utilizando la misma interfaz y no necesita conocer los cambios internos del proveedor.

Conclusión: Adapter permite aislar los cambios de las APIs externas, evitando que se propaguen al resto del sistema. Si cambia Stripe, solamente será necesario actualizar su adaptador, siempre que el contrato común `IPagoProcesador` siga siendo suficiente.

8.d. Integración con Factory Method, Proxy y Decorator

Para ampliar la solución anterior incorporaremos tres patrones de diseño, cada uno con una responsabilidad específica.

8.d.1. Factory Method: creación de adaptadores

Objetivo: encapsular la creación de los adaptadores para que el cliente no tenga que instanciar directamente `PayPalAdapter`, `BankTransferAdapter` o `StripeAdapter`.

Se define una clase creadora abstracta que declara el método `CrearProcesador()`. Después, cada creador concreto implementa ese método para construir el adaptador correspondiente.

Implementación en C#:

```
// Creador abstracto
public abstract class CreadorPago
{
    public abstract IPagoProcesador CrearProcesador();
}

// Creador de PayPal
public class CreadorPayPal : CreadorPago
{
    public override IPagoProcesador CrearProcesador()
    {
        return new PayPalAdapter(new PayPalOld());
    }
}

// Creador de transferencia bancaria
public class CreadorBanco : CreadorPago
{
    public override IPagoProcesador CrearProcesador()
    {
        return new BankTransferAdapter(new BankTransferOld());
    }
}

// Creador de Stripe
public class CreadorStripe : CreadorPago
{
    public override IPagoProcesador CrearProcesador()
    {
        return new StripeAdapter(new StripeModern());
    }
}
```

Ahora podemos seleccionar el creador según la configuración del sistema:

```
public static class FabricaPagos
{
    public static CreadorPago ObtenerCreador(string proveedor)
    {
        return proveedor.ToLower() switch
        {
            "paypal" => new CreadorPayPal(),
            "banco" => new CreadorBanco(),
            "stripe" => new CreadorStripe(),

            _ => throw new ArgumentException(
                "Proveedor de pago no válido.")
        };
    }
}
```

Ventaja: el cliente deja de responsabilizarse por la creación de los adaptadores. Si se incorpora un proveedor nuevo, basta con implementar su adaptador y su creador concreto.

8.d.2. Proxy: almacenamiento en caché de respuestas de pago

Objetivo: proporcionar un intermediario que controle el acceso al procesador real y almacene los resultados de operaciones previamente ejecutadas.

El Proxy implementará la misma interfaz `IPagoProcesador`, por lo que podrá sustituir al procesador original sin modificar el cliente.

Consideración importante: no debemos utilizar únicamente el monto como clave de caché. Dos compras diferentes pueden tener exactamente el mismo importe.

Para evitarlo, incorporaremos un identificador único de operación (`idOperacion`), que permitirá reconocer cuándo una solicitud corresponde a una transacción que ya fue procesada.

Por tanto, necesitamos ampliar ligeramente la interfaz original:

```
public interface IPagoProcesador
{
    bool Procesar(decimal monto, string idOperacion);
}
```

Los tres adaptadores deberán implementar esta nueva firma, manteniendo sus respectivas traducciones hacia las APIs externas.

Implementación del Proxy:

```
using System.Collections.Concurrent;

public class PagoProxy : IPagoProcesador
{
    private readonly IPagoProcesador _procesadorReal;

    private readonly ConcurrentDictionary<string,
        (decimal Monto, bool Resultado)> _cache = new();

    private readonly object _bloqueo = new();

    public PagoProxy(IPagoProcesador procesadorReal)
    {
        _procesadorReal = procesadorReal;
    }

    public bool Procesar(decimal monto, string idOperacion)
    {
        if (string.IsNullOrWhiteSpace(idOperacion))
            throw new ArgumentException(
                "El identificador de operación es obligatorio.");

        lock (_bloqueo)
        {
            if (_cache.TryGetValue(idOperacion, out var registro))
            {
                if (registro.Monto != monto)
                    throw new InvalidOperationException(
                        "El identificador ya fue utilizado con otro monto.");

                Console.WriteLine(
                    "[Proxy] Resultado recuperado de caché.");

                return registro.Resultado;
            }

            Console.WriteLine(
                "[Proxy] Ejecutando nueva operación de pago.");

```

```
            bool resultado = _procesadorReal.Procesar(
                monto, idOperacion);

            // Se almacenan los resultados exitosos.
            if (resultado)
            {
                _cache[idOperacion] = (monto, resultado);
            }

            return resultado;
        }
    }
}
```

Funcionamiento:

1. El cliente solicita procesar un pago proporcionando su identificador de operación.
2. El Proxy comprueba si existe un resultado almacenado para ese identificador.
3. Si existe, devuelve el resultado almacenado sin volver a ejecutar el pago.
4. Si no existe, delega la operación al adaptador correspondiente.
5. Si el pago tiene éxito, almacena el resultado para futuras solicitudes.

Esta implementación evita repetir una operación exitosa dentro de la misma instancia del Proxy.

En un sistema real, también sería necesario utilizar mecanismos de idempotencia del proveedor de pagos y almacenamiento persistente para protegerse frente a reinicios y fallos. La caché en memoria, por sí sola, no garantiza que nunca se duplique un cobro.

8.d.3. Decorator: auditoría de transacciones

Objetivo: incorporar el registro de auditoría de todas las solicitudes de pago sin modificar las clases de los adaptadores ni alterar su funcionamiento interno.

Decorator permite envolver un objeto que implementa `IPagoProcesador` con otro objeto que implementa la misma interfaz, añadiendo responsabilidades adicionales.

En nuestro caso, crearemos `PagoAuditoriaDecorator`.

Implementación en C#:

```
public class PagoAuditoriaDecorator : IPagoProcesador
{
    private readonly IPagoProcesador _procesador;
    private readonly IAuditoria _auditoria;

    public PagoAuditoriaDecorator(
        IPagoProcesador procesador,
        IAuditoria auditoria)
    {
        _procesador = procesador;
        _auditoria = auditoria;
    }

    public bool Procesar(decimal monto, string idOperacion)
    {
        try
        {
            bool resultado = _procesador.Procesar(
                monto, idOperacion);

```

```
            _auditoria.Registrar(
                idOperacion,
                monto,
                resultado,
                "Operación finalizada");

            return resultado;
        }
        catch (Exception ex)
        {
            _auditoria.Registrar(
                idOperacion,
                monto,
                false,
                $"Error: {ex.Message}");

            throw;
        }
    }
}
```

Definimos también una interfaz para el servicio de auditoría:

```
public interface IAuditoria
{
    void Registrar(
        string idOperacion,
        decimal monto,
        bool resultado,
        string detalle);
}

public class AuditoriaConsola : IAuditoria
{
    public void Registrar(
        string idOperacion,
        decimal monto,
        bool resultado,
        string detalle)
    {
        Console.WriteLine(
            $"[AUDITORÍA] Operación: {idOperacion} | " +
            $"Monto: {monto:F2} | " +
            $"Resultado: {resultado} | " +
            $"Detalle: {detalle}");
    }
}
```

De esta manera, cada solicitud que atraviesa el Decorator queda registrada, independientemente de si el Proxy ejecuta una nueva operación o devuelve un resultado almacenado en caché.

Para una auditoría real, el registro debería persistirse en una base de datos o almacenamiento duradero.

8.d.4. Integración de los tres patrones

Finalmente, combinamos los tres patrones para construir el sistema completo.

La secuencia de responsabilidades será:

1. **Factory Method:** selecciona y crea el adaptador según el proveedor configurado.
2. **Proxy:** controla el acceso al adaptador y gestiona la caché de resultados.

3. **Decorator:** registra la auditoría de cada solicitud.
4. **Adapter:** traduce la operación común a la interfaz específica del proveedor.
5. **Sistema Legacy o Moderno:** ejecuta el procesamiento real del pago.

El siguiente código muestra cómo construir esta composición:

```
public class SistemaComercio
{
    private readonly IPagoProcesador _procesadorPago;

    public SistemaComercio(IPagoProcesador procesadorPago)
    {
        _procesadorPago = procesadorPago;
    }

    public void RealizarCompra(
        decimal monto,
        string idOperacion)
    {
        Console.WriteLine(
            $"Procesando compra de ${monto:F2}");

        bool resultado = _procesadorPago.Procesar(
            monto, idOperacion);

        Console.WriteLine(
            resultado
                ? "Compra completada exitosamente."
                : "Error al procesar la compra.");
    }
}
```

Ejemplo de configuración:

```
public class Program
{
    public static void Main()
    {
        // 1. Selección del proveedor.
        CreadorPago creador =
            FabricaPagos.ObtenerCreador("stripe");

        // 2. Creación del adaptador.
        IPagoProcesador adaptador =
            creador.CrearProcesador();

        // 3. Incorporación del Proxy.
        IPagoProcesador proxy =
            new PagoProxy(adaptador);

        // 4. Incorporación de la auditoría.
        IAuditoria auditoria = new AuditoriaConsola();

        IPagoProcesador procesadorFinal =
            new PagoAuditoriaDecorator(proxy, auditoria);

        // 5. Uso desde el cliente.
        SistemaComercio comercio =
            new SistemaComercio(procesadorFinal);

        comercio.RealizarCompra(150.00m, "OP-001");

        // Se recupera el resultado de la caché.
```

```
30          comercio.RealizarCompra(150.00m, "OP-001");
31      }
32  }
```

Resultado esperado:

```
Procesando compra de $150.00
[Proxy] Ejecutando nueva operación de pago.
[Stripe Modern] Transacción realizada por $150.00 (USD).
[AUDITORÍA] Operación: OP-001 | Monto: 150.00 |
Resultado: True | Detalle: Operación finalizada

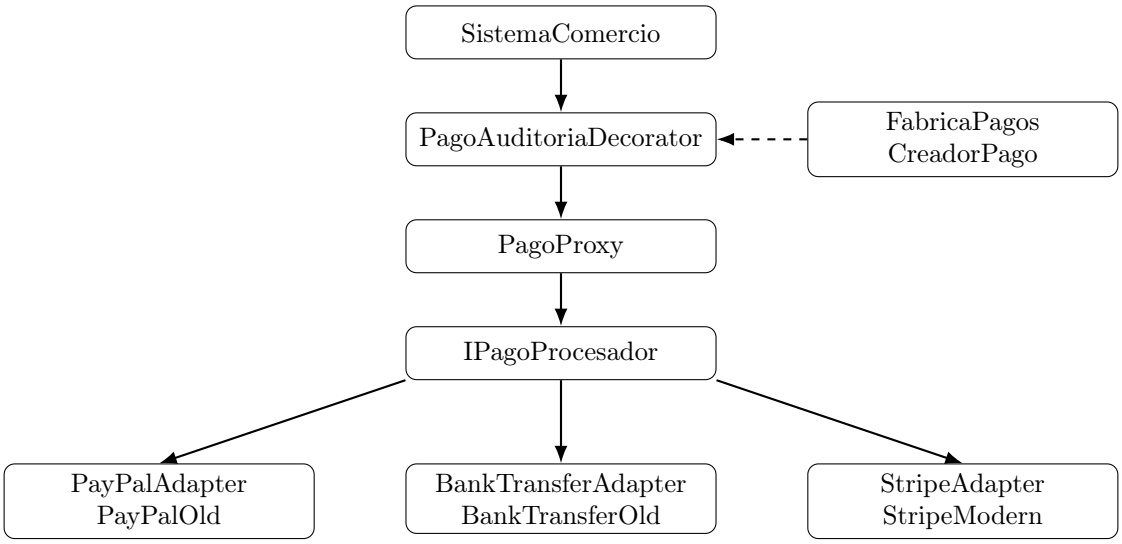
Compra completada exitosamente.

Procesando compra de $150.00
[Proxy] Resultado recuperado de caché.
[AUDITORÍA] Operación: OP-001 | Monto: 150.00 |
Resultado: True | Detalle: Operación finalizada

Compra completada exitosamente.
```

8.d.5. Relación entre los patrones utilizados

La estructura final puede representarse de la siguiente manera:



Cada patrón tiene una responsabilidad independiente:

Patrón	Responsabilidad dentro del sistema
Adapter	Unificar las interfaces incompatibles de los proveedores.
Factory Method	Crear el adaptador correspondiente según la configuración.
Proxy	Controlar el acceso al procesador y cachear resultados.
Decorator	Añadir auditoría sin modificar los adaptadores.

Conclusión general: La integración de estos patrones permite construir un sistema de procesamiento de pagos extensible y desacoplado. Adapter resuelve la incompatibilidad entre los proveedores, Factory Method centraliza su creación, Proxy controla las solicitudes y evita repetir operaciones exitosas, mientras que Decorator incorpora el registro de auditoría sin modificar las clases existentes.

Así, cada patrón cumple una responsabilidad específica y se complementan para facilitar el mantenimiento y la evolución del sistema.

Ejercicio 9 — Sistema de Notificaciones de Bolsa

9.a. Identificación del patrón, justificación y análisis de alternativas

9.a.1. Patrón seleccionado: Observer

El patrón más adecuado es Observer (también llamado Publicador–Suscriptor). Define una dependencia uno-a-muchos entre objetos: cuando el sujeto cambia de estado, todos sus dependientes son notificados y actualizados automáticamente.

9.a.2. Justificación

Cada característica del enunciado se corresponde con algo que el patrón resuelve directamente:

Característica del problema	**Lo que aporta Observer**
Notificar automáticamente a miles de clientes	Relación uno-a-muchos: cuando cambia el estado del sujeto (el precio), todos sus dependientes se enteran sin pedirlo.
Cuando ciertas acciones alcanzan precios	El cambio de estado del sujeto (`ActualizarPrecio`) es el evento que dispara `Notificar()`.
Los clientes quieren suscribirse a diferentes acciones	`Suscribir`/`Desuscribir` en tiempo de ejecución; un cliente puede observar varios sujetos a la vez.
Umbrales personalizados	Cada observador decide en su `Actualizar` si el cambio le interesa; el sujeto no sabe nada de los umbrales.
Notificaciones inmediatas	Modelo push: el aviso sale en el mismo momento del cambio, no en la próxima consulta.
Email, SMS y push	El canal es una responsabilidad aparte (`ICanalNotificacion`): se añaden canales sin tocar ni la acción ni el cliente.

La idea central es el bajo acoplamiento: la `Accion` solo conoce la interfaz `IObservadorAccion`. No sabe si quien la observa es un cliente, un módulo de auditoría o un gráfico en tiempo real, así que se pueden añadir observadores nuevos sin modificarla (principio Abierto/Cerrado).

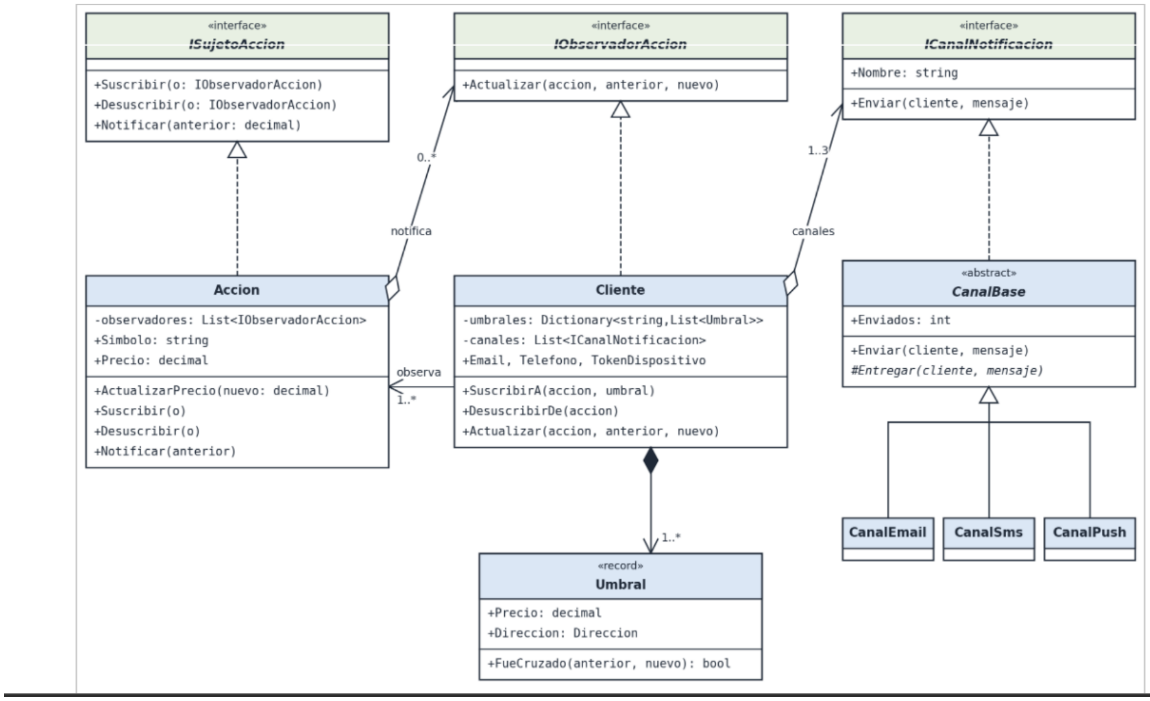
9.a.3. Clases que intervienen

Rol	**Clase**	**Responsabilidad**
Subject	`ISujetoAccion`	Contrato: `Suscribir`, `Desuscribir`, `Notificar`.
ConcreteSubject	`Accion`	Guarda `Simbolo`, `Precio` y la lista de observadores. Al cambiar el precio llama a `Notificar(precioAnterior)`.
Observer	`IObservadorAccion`	Contrato: `Actualizar(accion, precioAnterior, precioNuevo)`.
ConcreteObserver	`Cliente`	Guarda sus umbrales por símbolo y sus canales preferidos. En `Actualizar` comprueba si algún umbral fue cruzado y, si es así, envía el mensaje por sus canales.
Objeto valor	`Umbral`	Precio y dirección (Sube/Baja). `FueCruzado(anterior,nuevo)` evita avisos repetidos mientras el precio se queda del mismo lado.
Colaboradores	`ICanalNotificacion`, `CanalBase`, `CanalEmail`, `CanalSms`, `CanalPush`	Entregan el mensaje. Son intercambiables y no forman parte del Observer en sí.

9.a.4. Relaciones y diagrama de clases

- `Accion` implementa `ISujetoAccion`; `Cliente` implementa `IObservadorAccion`.
- `Accion` agrega 0..* observadores, pero solo a través de la interfaz: no conoce la clase `Cliente`.
- `Cliente` y `Accion` tienen una relación muchos-a-muchos: un cliente observa varias acciones y una acción tiene muchos clientes.
- `Cliente` compone sus umbrales (viven y mueren con él) y agrega los canales (instancias compartidas entre todos los clientes).
- `CanalEmail`, `CanalSms` y `CanalPush` heredan de `CanalBase`, que implementa `ICanalNotificacion`.

Diagrama de clases



9.a.5. Flujo de una notificación

1. El cliente llama a `SuscribirA(accion, umbral)`: guarda el umbral y, si es la primera vez para esa acción, se registra con `accion.Suscribir(this)`.
2. Llega un precio nuevo: `accion.ActualizarPrecio(p)` cambia el estado y llama a `Notificar(anterior)`.
3. `Notificar` recorre una copia de la lista de observadores (así nadie rompe la iteración si se desuscribe dentro de `Actualizar`) y llama a `Actualizar` en cada uno.
4. Cada `Cliente` filtra con sus umbrales; si alguno fue cruzado, envía el mensaje por cada canal que tiene configurado.

9.a.6. ¿Por qué no Mediator?

- La comunicación tiene un solo sentido. Mediator resuelve interacciones muchos-a-muchos entre colegas que se hablan entre sí (como los aviones del ejercicio 6). Aquí los clientes no se comunican entre ellos ni con la acción: solo reciben. Es una difusión uno-a-muchos, que es justo lo que hace Observer.

- Crea un "objeto dios". Un mediador tendría que conocer todas las acciones, todos los clientes, todos los umbrales y todos los canales, y concentraría la lógica de filtrado. Con miles de clientes se vuelve un cuello de botella y un punto donde cualquier cambio obliga a modificar la misma clase.
- Acopla más, no menos. El mediador tiene que conocer los tipos concretos de los colegas para coordinarlos. En Observer el sujeto solo conoce una interfaz de un método.
- En la práctica, un broker de eventos (Pub/Sub) es un Observer con un intermediario, no un Mediator: no coordina reglas de interacción, solo reenvía eventos por tema.

9.a.7. ¿Por qué no polling (consulta periódica)?

Con polling, cada cliente pregunta cada T segundos "¿cuál es el precio?":

- **Carga desperdiciada:** el número de consultas es clientes × acciones $/T$, cambie o no el precio. Con 1 000 clientes, 5 acciones y $T = 1$ s son 5 000 peticiones por segundo, y casi todas devuelven "nada nuevo". En Observer el trabajo es proporcional a los cambios reales.
- **No es inmediato:** la latencia en el peor caso es T. Para bajarla hay que reducir T, y eso aumenta la carga. Push no tiene ese compromiso.
- **Pierde eventos:** si el precio cruza el umbral y vuelve entre dos consultas, el cliente nunca se entera. El sujeto, en cambio, ve todas las transiciones (anterior → nuevo), así que `FueCruzado` detecta el cruce aunque dure un solo tick.
- **Duplica lógica:** cada cliente tendría que guardar el último precio visto y comparar por su cuenta.

9.b. Implementación con cinco acciones y mil clientes

9.b.1. Estructura

Archivo	Contenido
`Observer.cs`	`IObservadorAccion`, `ISujetoAccion`, `Accion`.
`Cliente.cs`	`Direccion`, `Umbral`, `Cliente`.
`Canales.cs`	`ICanalNotificacion`, `CanalBase`, `CanalEmail`, `CanalSms`, `CanalPush`.
`Program.cs`	Crea 5 acciones y 1 000 clientes y simula 60 ticks de mercado.

9.b.2. Cómo se cumplen los requisitos

- 5 acciones: AAPL, MSFT, GOOGL, AMZN, TSLA.
- 1 000 clientes, cada uno siguiendo entre 1 y 3 acciones elegidas al azar.
- Umbrales personalizados: de 1 a 2 por acción y cliente, entre ±1 % y ±8 % del precio inicial, de subida o de bajada.
- Email, SMS y push: cada cliente tiene un subconjunto no vacío de los tres canales (máscara de bits del 1 al 7).
- El mercado es un paseo aleatorio de ±2 % por tick con semilla fija (42), así que el resultado es reproducible.

9.b.3. Decisiones de diseño

- **El sujeto no filtra.** `Accion` notifica a todos y cada `Cliente` decide con sus umbrales. Es el Observer clásico: el sujeto queda simple y desacoplado. El coste es $O(n)$ observadores por cambio de precio; para 1M de clientes esto se optimiza indexando los umbrales (inciso d).
- **Cruce, no nivel.** Se avisa cuando el precio cruza el umbral (anterior $< u \leq$ nuevo), no en cada tick que pase por encima. Si el precio oscila alrededor del umbral, se avisa en cada cruce; por eso salen más alertas que umbrales.
- **Una sola suscripción por acción.** Aunque el cliente tenga varios umbrales en AAPL, se registra una sola vez en esa acción (`SuscribirA` lo comprueba) y no recibe `Actualizar` duplicados.
- **Notificación síncrona.** En esta versión `Notificar` llama a los observadores uno tras otro en el mismo hilo. Es lo que hace el patrón básico y es lo que hay que cambiar en el inciso c (cola o thread pool).

9.b.4. Código fuente

Observer.cs — Subject, Observer y ConcreteSubject

```
namespace NotificacionesBolsa;

// ===== Roles del patron Observer =====
public interface IObservadorAccion
{
    void Actualizar(Accion accion, decimal precioAnterior, decimal precioNuevo);
}

public interface ISujetoAccion
{
    void Suscribir(IObservadorAccion observador);
    void Desuscribir(IObservadorAccion observador);
    void Notificar(decimal precioAnterior);
}

public class Accion : ISujetoAccion
{
    private readonly List\textless{}IObservadorAccion\textgreater{} _observadores = new();
    public string Simbolo { get; }
    public decimal Precio { get; private set; }
    public int CantidadObservadores => _observadores.Count;

    public Accion(string simbolo, decimal precioInicial)
    {
        Simbolo = simbolo;
        Precio = precioInicial;
    }

    public void Suscribir(IObservadorAccion observador)
    {
        if (!_observadores.Contains(observador))
            _observadores.Add(observador);
    }

    public void Desuscribir(IObservadorAccion observador) =>
        _observadores.Remove(observador);

    public void ActualizarPrecio(decimal nuevoPrecio)
    {
        if (nuevoPrecio == Precio) return;
```

```
        var anterior = Precio;
        Precio = nuevoPrecio;
        Notificar(anterior);
    }

    public void Notificar(decimal precioAnterior)
    {
        foreach (var o in _observadores.ToArray())
            o.Actualizar(this, precioAnterior, Precio);
    }
}
```

Cliente.cs — ConcreteObserver y umbrales

```
namespace NotificacionesBolsa;

public enum Direccion { Sube, Baja }

public record Umbral(decimal Precio, Direccion Direccion)
{
    public bool FueCruzado(decimal anterior, decimal nuevo) => Direccion switch
    {
        Direccion.Sube => anterior < Precio && nuevo >= Precio,
        Direccion.Baja => anterior > Precio && nuevo <= Precio,
        _ => false
    };
}

public class Cliente : IObservadorAccion
{
    private readonly Dictionary<string, List<Umbral>> _umbrales = new();
    private readonly List\textless{}ICanalNotificacion\textgreater{} _canales;

    public int Id { get; }
    public string Nombre { get; }
    public string Email { get; }
    public string Telefono { get; }
    public string TokenDispositivo { get; }
    public int NotificacionesRecibidas { get; private set; }

    public Cliente(int id, string nombre, IEnumerable<ICanalNotificacion> canales)
    {
        Id = id;
        Nombre = nombre;
        Email = $"cliente{id}@mail.com";
        Telefono = $"+53 5{id:D7}";
        TokenDispositivo = $"dev-{id:X6}";
        _canales = canales.ToList();
    }

    public IReadOnlyList\textless{}ICanalNotificacion\textgreater{} Canales => _canales;

    public void SuscribirA(Accion accion, Umbral umbral)
    {
        if (!_umbrales.TryGetValue(accion.Simbolo, out var lista))
        {
            lista = new List<Umbral>();
            _umbrales[accion.Simbolo] = lista;
            accion.Suscribir(this);
        }
        lista.Add(umbral);
    }

```

```
    public void DesuscribirDe(Accion accion)
    {
        _umbrales.Remove(accion.Simbolo);
        accion.Desuscribir(this);
    }

    public void Actualizar(Accion accion, decimal precioAnterior, decimal precioNuevo)
    {
        if (!_umbrales.TryGetValue(accion.Simbolo, out var umbrales)) return;
        foreach (var u in umbrales.Where(u => u.FueCruzado(precioAnterior, precioNuevo)))
        {
            var verbo = u.Direccion == Direccion.Sube ? "subió" : "bajó";
            var mensaje = $"{accion.Simbolo} {verbo} a {precioNuevo:F2} (umbral {u.Precio:F2})";
            foreach (var canal in _canales)
                canal.Enviar(this, mensaje);
            NotificacionesRecibidas++;
        }
    }
}
```

Canales.cs — canales de notificación

```
namespace NotificacionesBolsa;

public interface ICanalNotificacion
{
    string Nombre { get; }
    int Enviados { get; }
    void Enviar(Cliente cliente, string mensaje);
}

public abstract class CanalBase : ICanalNotificacion
{
    public static int MaxMuestras { get; set; } = 15;
    private static int _impresas;
    public abstract string Nombre { get; }
    public int Enviados { get; private set; }

    public void Enviar(Cliente cliente, string mensaje)
    {
        Enviados++;
        var linea = Entregar(cliente, mensaje);
        if (_impresas++ < MaxMuestras)
            Console.WriteLine($"  {linea}");
    }

    protected abstract string Entregar(Cliente cliente, string mensaje);
}

public class CanalEmail : CanalBase
{
    public override string Nombre => "Email";
    protected override string Entregar(Cliente c, string m) => $"[EMAIL → {c.Email}] {m}";
}

public class CanalSms : CanalBase
{
    public override string Nombre => "SMS";
    protected override string Entregar(Cliente c, string m) => $"[SMS   → {c.Telefono}] {m}";
}

public class CanalPush : CanalBase
{
```

```
    public override string Nombre => "Push";
    protected override string Entregar(Cliente c, string m) => $"[PUSH  → {c.TokenDispositivo}] {
    m}";
}
```

Program.cs — simulación con 5 acciones y 1 000 clientes

```
using NotificacionesBolsa;

const int NumClientes = 1000;
const int NumTicks = 60;
var rnd = new Random(42); // semilla fija: resultados reproducibles

var acciones = new[]
{
    new Accion("AAPL", 230.00m),
    new Accion("MSFT", 420.00m),
    new Accion("GOOGL", 170.00m),
    new Accion("AMZN", 185.00m),
    new Accion("TSLA", 250.00m),
};

var email = new CanalEmail();
var sms = new CanalSms();
var push = new CanalPush();
ICanalNotificacion[] todosLosCanales = [email, sms, push];

var clientes = new List<Cliente>();
for (int id = 1; id <= NumClientes; id++)
{
    int mascara = rnd.Next(1, 8);
    var canales = todosLosCanales.Where((_, i) => (mascara & (1 << i)) != 0);
    var cliente = new Cliente(id, $"Cliente {id}", canales);

    foreach (var accion in acciones.OrderBy(_ => rnd.Next()).Take(rnd.Next(1, 4)))
    {
        for (int k = rnd.Next(1, 3); k > 0; k--)
        {
            var dir = rnd.Next(2) == 0 ? Direccion.Sube : Direccion.Baja;
            var porcentaje = (decimal)(0.01 + rnd.NextDouble() * 0.07);
            var precio = dir == Direccion.Sube
                ? accion.Precio * (1 + porcentaje)
                : accion.Precio * (1 - porcentaje);
            cliente.SuscribirA(accion, new Umbral(Math.Round(precio, 2), dir));
        }
    }
    clientes.Add(cliente);
}

Console.WriteLine("=== Suscripciones ===");
foreach (var a in acciones)
    Console.WriteLine($"  {a.Simbolo,-6} {a.Precio,8:F2}  observadores: {a.CantidadObservadores}")
    ;

Console.WriteLine($"\n=== Mercado: {NumTicks} ticks (primeras {CanalBase.MaxMuestras} entregas)
    ===");
for (int t = 0; t < NumTicks; t++)
{
    foreach (var a in acciones)
    {
        var variacion = (decimal)((rnd.NextDouble() * 2 - 1) * 0.02);
        a.ActualizarPrecio(Math.Round(a.Precio * (1 + variacion), 2));
    }
```

```
}

Console.WriteLine("\n=== Resumen ===");
foreach (var a in acciones)
    Console.WriteLine($"  {a.Simbolo,-6} precio final {a.Precio,8:F2}");
Console.WriteLine($"  Alertas disparadas: {clientes.Sum(c => c.NotificacionesRecibidas)}");
Console.WriteLine($"  Clientes notificados al menos una vez: {clientes.Count(c => c.
    NotificacionesRecibidas > 0)} / {NumClientes}");
foreach (var canal in todosLosCanales)
    Console.WriteLine($"  Entregas por {canal.Nombre,-5}: {canal.Enviados}");

var c1 = clientes[0];
foreach (var a in acciones) c1.DesuscribirDe(a);
Console.WriteLine($"\n  Cliente 1 desuscrito. Observadores de {acciones[0].Simbolo}: {acciones[0].
    CantidadObservadores}");
```

9.b.5. Salida de la ejecución

Resultado de `dotnet run` (solo se imprimen las primeras 15 entregas para no llenar la consola):

```
=== Suscripciones ===
AAPL     230.00  observadores: 417
MSFT     420.00  observadores: 366
GOOGL    170.00  observadores: 418
AMZN     185.00  observadores: 399
TSLA     250.00  observadores: 403

=== Mercado: 60 ticks (primeras 15 entregas) ===
[EMAIL → cliente450@mail.com] AAPL bajó a 227.50 (umbral 227.63)
[SMS   → +53 50000450] AAPL bajó a 227.50 (umbral 227.63)
[PUSH  → dev-0001C2] AAPL bajó a 227.50 (umbral 227.63)
[EMAIL → cliente459@mail.com] AAPL bajó a 227.50 (umbral 227.58)
[EMAIL → cliente771@mail.com] AAPL bajó a 227.50 (umbral 227.60)
[SMS   → +53 50000771] AAPL bajó a 227.50 (umbral 227.60)
[PUSH  → dev-000303] AAPL bajó a 227.50 (umbral 227.60)
[SMS   → +53 50000777] AAPL bajó a 227.50 (umbral 227.58)
[EMAIL → cliente855@mail.com] AAPL bajó a 227.50 (umbral 227.57)
[EMAIL → cliente990@mail.com] AAPL bajó a 227.50 (umbral 227.50)
[PUSH  → dev-0003DE] AAPL bajó a 227.50 (umbral 227.50)
[EMAIL → cliente12@mail.com] GOOGL subió a 173.32 (umbral 172.05)
[SMS   → +53 50000012] GOOGL subió a 173.32 (umbral 172.05)
[PUSH  → dev-00000C] GOOGL subió a 173.32 (umbral 172.05)
[PUSH  → dev-000011] GOOGL subió a 173.32 (umbral 172.36)

=== Resumen ===
AAPL   precio final   220.17
MSFT   precio final   344.13
GOOGL  precio final   191.44
AMZN   precio final   173.19
TSLA   precio final   269.21
Alertas disparadas: 4575
Clientes notificados al menos una vez: 923 / 1000
Entregas por Email: 2477
Entregas por SMS  : 2608
Entregas por Push : 2610

Cliente 1 desuscrito. Observadores de AAPL: 416
```

9.c. Gestión de fallos, ejecución asíncrona y comportamiento de Observer

9.c.1. Las tres preguntas del inciso

1. ¿Qué pasa si un cliente no puede ser notificado?
2. ¿Cómo evitar bloquear el hilo principal?
3. ¿Observer es síncrono o asíncrono?

Antes de responder, recordemos la versión clásica de `Notificar()` heredada de b): si el cliente número 3 de 1000 tarda 30 segundos o lanza una excepción, el `foreach` se bloquea o se interrumpe y los clientes restantes nunca son notificados.

9.c.2. ¿Observer es síncrono o asíncrono?

El patrón Observer, tal como lo define el GoF, no dicta si la notificación es síncrona o asíncrona: solo define la relación uno-a-muchos y el mecanismo de suscripción/notificación. Sin embargo, la implementación clásica (la de b)) es síncrona: el Subject recorre la lista y llama a `Actualizar()` de cada observador en el mismo hilo que cambió el precio.

Conclusión: Observer es síncrono por defecto, pero puede (y en este problema debe) implementarse de forma asíncrona desacoplando la notificación del cambio de estado.

Listing 1: Observer clásico (síncrono): el problema heredado de b)

```
public interface IObservadorAccion
{
    void Actualizar(Accion accion, decimal precioAnterior, decimal precioNuevo);
}

public class Accion
{
    private readonly List\textless{}IObservadorAccion\textgreater{} _observadores = new();
    public string Simbolo { get; }
    public Accion(string simbolo) { Simbolo = simbolo; }

    public void ActualizarPrecio(decimal precio)
    {
        foreach (var o in _observadores)
        {
            // Bloquea y puede lanzar excepción.
            o.Actualizar(this, Precio, precio);
        }
    }
}
```

9.c.3. Problemas de la versión síncrona

1. **Bloqueo:** el hilo que recibe el precio de mercado queda esperando a que termine cada envío de email o SMS.
2. **Fallo en cascada:** si un observador lanza una excepción, el `foreach` se interrumpe y los clientes restantes nunca son notificados.
3. **Acoplamiento temporal:** la velocidad del sistema la dicta el observador más lento.
4. **Concurrencia:** si otro hilo llama a `Suscribir()` mientras se itera la lista, C# lanza `InvalidOperationException` (colección modificada).

9.c.4. ¿Qué pasa si un cliente no puede ser notificado?

Un fallo de notificación es una situación normal, no excepcional (número apagado, correo inválido, servicio caído). El diseño debe:

1. Aislar el fallo: un cliente que falla no afecta a los demás (`try/catch` por observador).
2. Reintentar con espera creciente (backoff exponencial) un número limitado de veces.
3. Persistir la notificación si agota los reintentos (cola de fallidas o dead-letter queue) para reenviarla después. Esto es lo que exige la restricción at-least-once.
4. Registrar el error (log) para auditoría.

9.c.5. ¿Cómo evitar bloquear el hilo principal?

La idea clave: el Subject no debe ejecutar las notificaciones, solo encolarlas. Un grupo de trabajadores en segundo plano las consume. Esto convierte la notificación en un esquema productor–consumidor. En C# la herramienta natural es `System.Threading.Channels`.

Listing 2: Observador y notificación asíncronos (evolución de la interfaz de b)

```
public interface IObservadorAccion
{
    // Firma síncrona original de b).
    void Actualizar(Accion accion, decimal precioAnterior, decimal precioNuevo);
    // Nueva variante no bloqueante.
    Task ActualizarAsync(Accion accion, decimal precioAnterior, decimal precioNuevo);
}

public record Notificacion(
    IObservadorAccion Cliente, string Simbolo,
    decimal PrecioAnterior, decimal PrecioNuevo);

// Par cliente-umbral que permite indexar por umbral (ver inciso d).
public class Suscripcion
{
    public IObservadorAccion Cliente { get; }
    public Umbral Umbral { get; }
    public Suscripcion(IObservadorAccion cliente, Umbral umbral)
    {
        Cliente = cliente;
        Umbral = umbral;
    }
    // Se conserva la semántica de b): cruce, no nivel.
    public bool FueCruzado(decimal anterior, decimal nuevo) =>
        Umbral.FueCruzado(anterior, nuevo);
}
```

Listing 3: Despachador asíncrono con reintentos y cola de fallidas

```
using System.Collections.Concurrent;
using System.Threading.Channels;

public class DespachadorAsincrono
{
    private readonly Channel<Notificacion> _canal =
        Channel.CreateUnbounded<Notificacion>();
    // En producción esta cola se persiste en base de datos o disco.
    private readonly ConcurrentQueue<Notificacion> _fallidas = new();
    private const int MaxReintentos = 3;

    // Lo llama el Subject: es instantáneo, no espera al envío.
```

```
    public void Encolar(Notificacion n) => _canal.Writer.TryWrite(n);

    // Arranca N trabajadores en segundo plano.
    public Task Iniciar(int trabajadores, CancellationToken ct)
    {
        var tareas = new List<Task>();
        for (int i = 0; i < trabajadores; i++)
            tareas.Add(Task.Run(() => ProcesarAsync(ct), ct));
        return Task.WhenAll(tareas);
    }

    private async Task ProcesarAsync(CancellationToken ct)
    {
        await foreach (var n in _canal.Reader.ReadAllAsync(ct))
        {
            bool entregada = false;
            for (int intento = 1; intento <= MaxReintentos && !entregada; intento++)
            {
                try
                {
                    await n.Cliente.ActualizarAsync(
                        new Accion(n.Simbolo), n.PrecioAnterior, n.PrecioNuevo);
                    entregada = true;
                }
                catch (OperationCanceledException) when (ct.IsCancellationRequested)
                {
                    throw; // cancelación cooperativa: no es un fallo de envío.
                }
                catch (Exception ex)
                {
                    // Backoff exponencial: 200 ms, 400 ms, 800 ms.
                    Console.Error.WriteLine(
                        $"Fallo intento {intento} para {n.Simbolo}: {ex.Message}");
                    await Task.Delay(200 * (int)Math.Pow(2, intento - 1), ct);
                }
            }
            if (!entregada) _fallidas.Enqueue(n); // persistir para reenviar.
        }
    }
}
```

Listing 4: Accion evolucionada: solo evalúa y encola

```
public class Accion : ISujetoAccion
{
    private readonly List<Suscripcion> _suscripciones = new();
    private readonly object _candado = new object();
    private readonly DespachadorAsincrono _despachador;
    private readonly List\textless{}IObservadorAccion\textgreater{} _observadores = new();

    public string Simbolo { get; }
    public decimal Precio { get; private set; }

    public Accion(string simbolo, DespachadorAsincrono despachador)
    {
        Simbolo = simbolo;
        _despachador = despachador;
    }

    public void Suscribir(IObservadorAccion o)
    {
        lock (_candado)
            if (!_observadores.Contains(o)) _observadores.Add(o);
    }

```

```
    public void Desuscribir(IObservadorAccion o)
    {
        lock (_candado) _observadores.Remove(o);
    }

    public void ActualizarPrecio(decimal nuevoPrecio)
    {
        if (nuevoPrecio == Precio) return;
        var anterior = Precio;
        Precio = nuevoPrecio;
        Notificar(anterior);
    }

    public void Notificar(decimal precioAnterior)
    {
        Suscripcion[] copia;
        lock (_candado) { copia = _suscripciones.ToArray(); }
        // Instantánea segura; solo se evalúa el umbral y se encola.
        foreach (var s in copia)
        {
            if (s.FueCruzado(precioAnterior, Precio))
                _despachador.Encolar(new Notificacion(
                    s.Cliente, Simbolo, precioAnterior, Precio));
        }
    }
}
```

9.c.6. Qué se gana con este cambio

- `ActualizarPrecio` solo evalúa umbrales y encola: el hilo principal queda libre de inmediato.
- Copiar la lista bajo `lock` evita la excepción por modificación concurrente sin mantener el candado durante el envío.
- El `try/catch` dentro del trabajador aísla cada fallo.
- El Subject sigue sin conocer las clases concretas de los observadores (email, SMS, push): solo ve `IObservadorAccion`.

9.c.7. Errores comunes

- Usar `async void` para notificar: las excepciones no se pueden capturar.
- Lanzar `Task.Run` sin control para cada notificación: con muchos clientes satura el thread pool (ver inciso d).
- Mantener el `lock` mientras se notifica: reintroduce el bloqueo que se quería evitar.

9.c.8. Resumen del inciso c)

Cliente no notificable: se aísla el fallo, se reintenta con backoff y, si persiste, se guarda en una cola de fallidas para reenvío posterior. No bloquear el hilo principal: el Subject solo encola; trabajadores en segundo plano (Channel + Task) envían. ¿Síncrono o asíncrono?: el Observer clásico es síncrono, pero el patrón admite una variante asíncrona, que es la adecuada aquí.

9.d. Rendimiento con un millón de clientes y escalabilidad

9.d.1. Planteamiento del problema

Si ahora hay $N = 10^6$ clientes suscritos a una acción:

1. ¿Cuánto cuesta cada cambio de precio con el Observer clásico?
2. ¿Cómo cambia el orden de complejidad al usar un thread pool o Reactive Streams?

9.d.2. Modelo de costo

Definimos:

- N: número de observadores suscritos a la acción.
- $K \leq N$: número de observadores cuyo umbral se cruza en este cambio (coherente con `FueCruzado` de b).
- t: tiempo promedio de una notificación (red, E/S). Suponemos $t = 50$ ms.

El costo de comprobar un umbral es una simple comparación (nanosegundos), así que el cuello de botella es la notificación, no el recorrido de la lista.

9.d.3. Observer clásico síncrono

$$T_{\text{sinc}} = O(N) + K \cdot t$$

En el peor caso ($K = N = 10^6$):

$$T_{\text{sinc}} \approx 10^6 \times 0{,}05 \text{ s} = 5 \times 10^4 \text{ s} \approx 13{,}9 \text{ horas.}$$

Es inviable: una notificación "inmediata" llegaría al último cliente casi 14 horas después. Además, mientras tanto el hilo principal estaría bloqueado.

9.d.4. Observer con Thread Pool

Con P trabajadores en paralelo, el trabajo total sigue siendo $O(K)$ notificaciones, pero el tiempo de reloj se divide aproximadamente entre P:

$$T_{\text{pool}} \approx O(N) + \frac{K \cdot t}{P}.$$

Con $P = 100$ y $K = 10^6$:

$$T_{\text{pool}} \approx \frac{10^6 \times 0{,}05}{100} = 500 \text{ s} \approx 8{,}3 \text{ minutos.}$$

Importante: el orden de complejidad del trabajo total no cambia ($O(N)$), pero la latencia baja en un factor P. El límite práctico lo ponen los hilos disponibles y el ancho de banda del proveedor de email/SMS.

Listing 5: Thread pool con paralelismo acotado

```
public class NotificadorConPool
{
    private readonly int _gradoParalelismo;
    public NotificadorConPool(int gradoParalelismo) =>
```

```
        _gradoParalelismo = gradoParalelismo;

    public async Task NotificarAsync(
        IEnumerable<Notificacion> pendientes, CancellationToken ct)
    {
        var opciones = new ParallelOptions
        {
            MaxDegreeOfParallelism = _gradoParalelismo,
            CancellationToken = ct
        };
        await Parallel.ForEachAsync(pendientes, opciones, async (n, token) =>
        {
            try
            {
                await n.Cliente.ActualizarAsync(
                    new Accion(n.Simbolo), n.PrecioAnterior, n.PrecioNuevo);
            }
            catch (Exception ex)
            {
                // Reintento / cola de fallidas como en el inciso c).
                Console.Error.WriteLine($"Fallo {n.Simbolo}: {ex.Message}");
            }
        });
    }
}
```

La pista es que, como `ActualizarAsync` usa `await` sobre E/S de red, el hilo no se queda bloqueado durante el envío: puede atender otras notificaciones. Por eso, con E/S asíncrona, se pueden tener miles de notificaciones en vuelo con pocos hilos reales.

9.d.5. Reducir N a K con un índice de umbrales

Recorrer a los 10^6 suscriptores en cada cambio de precio es $O(N)$ aunque solo unos pocos cumplan el umbral. Si se mantienen ordenados por umbral, los que se activan en una condición precio $\geq$ umbral son un prefijo de la lista, localizable con búsqueda binaria:

$$T_{\text{búsqueda}} = O(\log N + K).$$

Listing 6: Índice de umbrales para evitar recorrer N

```
public class IndiceUmbrales
{
    // Ordenada ascendentemente por Umbral.Precio.
    private readonly List<Suscripcion> _ordenadas = new();
    private readonly object _candado = new object();

    public void Agregar(Suscripcion s)
    {
        lock (_candado)
        {
            int pos = _ordenadas.FindIndex(
                x => x.Umbral.Precio > s.Umbral.Precio);
            if (pos < 0) _ordenadas.Add(s);
            else _ordenadas.Insert(pos, s);
        }
    }

    // Devuelve solo las K suscripciones con Precio <= precio.
    public List<Suscripcion> Disparadas(decimal precio)
    {
        lock (_candado)
        {
```

```
            int lo = 0, hi = _ordenadas.Count;
            while (lo < hi)
            {
                int mid = (lo + hi) / 2;
                if (_ordenadas[mid].Umbral.Precio <= precio) lo = mid + 1;
                else hi = mid;
            }
            return _ordenadas.GetRange(0, lo);
        }
    }
}
```

9.d.6. Reactive Streams

En Reactive Streams los precios son un flujo de datos y cada cliente se suscribe a ese flujo con operadores (`Where`, `Buffer`, `ObserveOn`). Aportan dos ventajas sobre un pool manual:

- **Backpressure:** si los consumidores son lentos, el sistema regula la velocidad del productor en vez de acumular memoria sin límite.
- **Composición:** filtrar, agrupar, reintentar y cambiar de hilo se expresan de forma declarativa.

En .NET se dispone de System.Reactive (Rx.NET), basado en `IObservable<T>/IObserver<T>`, y de `System.Threading.Channels` acotados para el backpressure.

Listing 7: Observer reactivo con Rx.NET (paquete NuGet System.Reactive)

```
using System.Reactive.Concurrency;
using System.Reactive.Linq;
using System.Reactive.Subjects;

public class AccionReactiva
{
    private readonly Subject<decimal> _precios = new();

    public void Suscribir(IObservadorAccion cliente, string simbolo, decimal umbral)
    {
        _precios
            .Where(p => p >= umbral) // solo si cumple el umbral
            .ObserveOn(TaskPoolScheduler.Default) // fuera del hilo principal
            .Subscribe(
                p => cliente.ActualizarAsync(new Accion(simbolo), p, p),
                ex => Console.WriteLine("Error: " + ex.Message));
    }

    public void ActualizarPrecio(decimal precio)
    {
        _precios.OnNext(precio); // el hilo principal solo publica
    }
}
```

Listing 8: Backpressure con un canal acotado

```
var opciones = new BoundedChannelOptions(capacity: 10_000)
{
    FullMode = BoundedChannelFullMode.Wait
};
var canal = Channel.CreateBounded<Notificacion>(opciones);

// Productor (Subject).
await canal.Writer.WriteAsync(notificacion); // se suspende si no hay espacio.
```

Rx con 10^6 suscripciones. En el ejemplo cada cliente genera su propia cadena `Where ->Subscribe`, por lo que cada precio sigue evaluándose N veces. Para 10^6 clientes es preferible combinar Rx con el índice de umbrales del apartado anterior (publicar únicamente a los K disparados).

9.d.7. Tabla comparativa: cómo cambia el orden de complejidad

El trabajo total sigue siendo lineal: hay que notificar a K clientes y eso no se puede evitar. Lo que cambia es la latencia: de $K \cdot t$ a $K \cdot t/P$. El índice de umbrales mejora la fase de búsqueda: de $O(N)$ a $O(\log N + K)$.

Enfoque	**Trabajo total**	**Latencia aprox.**
Observer síncrono	$O(N + K)$	$O(N + K \cdot t) \approx 13{,}9$ horas.
Observer + Thread Pool ($P = 100$)	$O(N + K)$	$O(N + K \cdot t/P) \approx 8{,}3$ minutos.
Pool + índice de umbrales	$O(\log N + K)$	$O(\log N + K \cdot t/P)$.
Reactive Streams	$O(N + K)$ (o $O(\log N + K)$ con índice)	Similar al pool.

Con P fijo el crecimiento sigue siendo lineal en K, solo con una constante menor. Para reducirlo más hay que aumentar P (más hilos, más nodos) o particionar los clientes en varios servidores.

Con 10^6 clientes la memoria también importa: una cola sin límite puede agotarla, por eso se usa un canal acotado (backpressure).

9.d.8. Resumen del inciso d)

Con 10^6 clientes el Observer síncrono tarda del orden de 13,9 horas en el peor caso (con $t = 50$ ms). Observer con Thread Pool reduce la latencia por un factor P (~ 8 minutos con $P = 100$) sin cambiar el trabajo total $O(N)$. Un índice de umbrales baja la búsqueda a $O(\log N + K)$, y Reactive Streams añaden backpressure y composición. La solución recomendada es combinar índice de umbrales + cola acotada + trabajadores asíncronos, con la cola de fallidas del inciso c) para cumplir at-least-once.